

```

# =====
# ===== EXERCICE 1 =====
# =====

import tensorflow as tf
from tensorflow import keras
import numpy as np

# QUESTION 1: VERSION TENSORFLOW
print("TensorFlow version:", tf.__version__)

# QUESTION 2: VERSION KERAS
print("Keras version:", keras.__version__)

# QUESTION 3: GPU DISPONIBLE ?
gpu = tf.config.list_physical_devices('GPU')
print("GPU disponible :", gpu)

# QUESTION 4: EXPLICATION

# Le GPU (Graphics Processing Unit) est un processeur spécialisé
# dans les calculs parallèles, particulièrement utile pour entraîner
# des modèles de deep learning.

# L'absence de GPU signifie que l'entraînement se fait uniquement sur
# le CPU,
# ce qui rend les calculs plus lents, surtout pour des modèles
# complexes
# ou de grandes quantités de données.

# Cependant, pour un modèle simple comme une régression linéaire avec
# peu de données,
# l'impact est très faible et le temps de calcul reste rapide même sans
# GPU.
# =====
# ===== EXERCICE 2 =====
# =====
# DATASET
surface = np.array([50, 80, 100, 120, 150], dtype=float)
prix = np.array([100, 160, 200, 240, 300], dtype=float)

# NORMALISATION
surface_norm = surface / max(surface)
prix_norm = prix / max(prix)

# MODEL
model = keras.Sequential([
    keras.layers.Dense(units=1, input_shape=[1])
])

```

```

# COMPILATION (Adam par défaut)
model.compile(optimizer='adam', loss='mean_squared_error')

# ENTRAÎNEMENT
model.fit(surface_norm, prix_norm, epochs=100, verbose=0)

# LOSS FINALE
loss = model.evaluate(surface_norm, prix_norm, verbose=0)
print("Loss finale :", loss)

# ===== EXPLICATIONS =====

# 1) Définition de la MSE (Mean Squared Error)
# La MSE est une fonction de perte qui mesure la moyenne des carrés des
erreurs
# entre les valeurs réelles (y_true) et les valeurs prédites (y_pred).
# Formule mathématique :
#  $MSE = (1/n) * \sum (y\_true - y\_pred)^2$ 

# 2) Pourquoi une MSE de 2.317 est anormalement élevée ?
# Dans ce problème, les données sont normalisées entre 0 et 1.
# Donc les erreurs possibles sont petites (entre 0 et 1).
# Comme les données sont comprises entre 0 et 1,
# une MSE proche de 1 indique une erreur très importante donc un modèle
mal entraîné
# Donc une valeur comme 2.317 est clairement anormale et indique que le
modèle apprend mal.

# 3) Quelle valeur serait acceptable ?
# Une bonne MSE dans ce cas devrait être proche de 0,
# par exemple inférieure à 0.01 pour un modèle bien entraîné.

# 4) Hypothèse sur le problème
# Le modèle ne converge pas correctement.
# Cela peut être dû à :
# - un learning rate inadapté (trop faible avec Adam par défaut)
# - un nombre d'époques insuffisant (100 epochs)
# Résultat : le modèle n'a pas le temps ou la vitesse pour bien
apprendre.

# ===== EXERCICE 3 =====

# EXTRACTION DES POIDS
weights = model.get_weights()

w = weights[0][0][0]

```

```

b = weights[1][0]

print("Poids w =", w)
print("Biais b =", b)

# =====
# EQUATION NORMALISEE
# =====

print("Equation normalisée :")
print("prix_norm =", w, "* surface_norm +", b)

# =====
# DENORMALISATION
# =====

# Rappel :
# surface_norm = surface / max(surface)
# prix_norm = prix / max(prix)

max_surface = max(surface)
max_prix = max(prix)

# On remplace surface_norm dans l'équation :
# prix_norm = w * (surface / max_surface) + b

# Puis on revient à l'échelle réelle :
# prix = prix_norm * max_prix

# Donc :
# prix = (w * surface / max_surface + b) * max_prix

# On développe :
w_real = (w * max_prix) / max_surface
b_real = b * max_prix

print("Equation dénormalisée :")
print("Prix =", w_real, "* Surface +", b_real)

# =====
# VERIFICATION POUR 100 m²
# =====

surface_test = 100

# Calcul manuel
prix_pred = w_real * surface_test + b_real

print("Prix prédit pour 100 m² :", prix_pred)

```

```

# Valeur réelle attendue = 200
# Le modèle donne une équation imparfaite,
# ce qui entraîne des prédictions qui ne correspondent pas exactement
aux valeurs réelles.
# Cela montre que le modèle n'a pas encore complètement convergé.
# =====
# ===== EXERCICE 4 =====
# =====

# PREDICTIONS
predictions = model.predict(surface_norm, verbose=0)
print("Prédictions :", predictions.flatten())
print("Valeurs réelles :", prix_norm)

# =====
# ANALYSE DES RESULTATS
# =====

# Les prédictions sont éloignées des valeurs réelles,
# ce qui montre que le modèle n'a pas encore correctement appris la
relation entre les variables.

# Le coefficient (w) n'est pas encore optimal,
# ce qui montre que la relation entre les variables n'est pas
correctement apprise.
# =====
# PARAMETRES RESPONSABLES
# =====

# Les deux paramètres responsables sont :

# 1) Le learning rate (taux d'apprentissage)
# La valeur par défaut de Adam (0.001) est trop faible ici,
# ce qui rend l'apprentissage très lent.

# 2) Le nombre d'époques (100)
# Ce nombre est insuffisant pour permettre au modèle de converger.

# =====
# RELATION LEARNING RATE / CONVERGENCE
# =====

# Le learning rate contrôle la vitesse d'apprentissage :
# - Trop petit → apprentissage très lent → pas de convergence
# - Trop grand → instabilité (oscillations)

# Un bon learning rate permet d'atteindre rapidement une solution
stable.

```

```

# =====
# DEFINITION DE LA CONVERGENCE
# =====

# La convergence signifie que le modèle atteint une solution stable
# où la loss devient minimale et ne change presque plus.

# La loss permet de détecter la convergence :
# - Si elle diminue progressivement puis se stabilise → convergence
# - Si elle reste élevée ou incohérente → problème d'apprentissage
# Le coefficient w n'est pas encore optimal, ce qui confirme que le
# modèle n'a pas correctement appris la relation.

#=====
# ===== EXERCICE 5 =====
# =====

# NOUVEAU MODELE CORRIGE
model2 = keras.Sequential([
    keras.layers.Dense(units=1, input_shape=[1])
])

# Optimizer avec learning rate modifié
optimizer = keras.optimizers.Adam(learning_rate=0.1)

model2.compile(optimizer=optimizer, loss='mean_squared_error')

# ENTRAINEMENT AVEC PLUS D'EPOCHS
for epoch in range(500):
    model2.fit(surface_norm, prix_norm, epochs=1, verbose=0)

    if epoch % 100 == 0:
        loss = model2.evaluate(surface_norm, prix_norm, verbose=0)
        print(f"Epoch {epoch} - Loss: {loss}")

# LOSS FINALE
final_loss = model2.evaluate(surface_norm, prix_norm, verbose=0)
print("Loss finale :", final_loss)

# =====
# EXPLICATIONS
# =====

# 1) Pourquoi learning rate = 0.1 ?
# Un learning rate plus élevé permet d'accélérer l'apprentissage.
# Ici, la valeur par défaut (0.001) était trop faible,
# ce qui empêchait le modèle de converger correctement.

```

```
# Avec 0.1, le modèle apprend plus rapidement et atteint une solution
correcte.
```

```
# 2) Pourquoi augmenter le nombre d'époques ?
# Plus d'époques signifie plus d'itérations d'apprentissage.
# Cela permet au modèle d'ajuster progressivement ses poids
# jusqu'à atteindre une solution optimale.
```

```
# 3) Evolution de la loss
# Au début (epoch 0), la loss est élevée.
# Puis elle diminue progressivement à mesure que le modèle apprend.
# Vers la fin (epoch 500), la loss devient très proche de 0,
# ce qui indique que le modèle a convergé correctement.
# =====
# EQUATION FINALE DU MODELE CORRIGE
# =====
```

```
weights2 = model2.get_weights()
```

```
w2 = weights2[0][0][0]
```

```
b2 = weights2[1][0]
```

```
print("Poids final w =", w2)
```

```
print("Biais final b =", b2)
```

```
print("Equation finale normalisée :")
```

```
print("prix_norm =", w2, "* surface_norm +", b2)
```

```
# =====
# ===== EXERCICE 6 =====
# =====
```

```
# 1) Pourquoi ce résultat est attendu ?
# Les données fournies suivent une relation parfaitement linéaire :
# Prix = 2 × Surface
# Après normalisation, cette relation devient :
# prix_norm = surface_norm
# Le modèle utilisé est une régression linéaire (une seule couche
Dense),
# donc il est parfaitement adapté pour apprendre ce type de relation.
# Il retrouve donc exactement l'équation attendue.
```

```
# 2) Signification d'une MSE  $\approx 0$ 
# Une MSE proche de 0 signifie que les prédictions du modèle
# sont quasiment identiques aux valeurs réelles.
# Le modèle a donc parfaitement appris les données.
```

```
# Cependant, en pratique, une MSE nulle est très rare,
# car les données réelles contiennent du bruit et des erreurs.
```

```

# 3) Limites du modèle
# Ce modèle est limité pour des données immobilières réelles car :
# - le prix ne dépend pas uniquement de la surface
# - la relation n'est pas strictement linéaire
# - il existe du bruit (variabilité des données)

# 4) Améliorations possibles
# - Ajouter d'autres variables explicatives (localisation, nombre de
pièces, etc.)
# - Utiliser un modèle plus complexe (réseau avec plusieurs couches ou
neurones)
# =====
# ===== EXERCICE 7 =====
# =====

# ===== RAPPORT DE SYNTHESE =====

# 1) Normalisation des données
# La normalisation consiste à ramener les variables dans un intervalle
réduit (ici entre 0 et 1).
# Dans ce TP, elle a été réalisée en divisant chaque variable par sa
valeur maximale.
# Elle permet d'améliorer la stabilité et la vitesse d'apprentissage du
modèle.

# 2) Fonction de perte MSE
# La MSE (Mean Squared Error) mesure l'écart moyen au carré entre les
valeurs réelles et les prédictions :
#  $MSE = (1/n) * \sum (y_{true} - y_{pred})^2$ 
# Une MSE élevée indique un mauvais apprentissage (cas du modèle
initial),
# tandis qu'une MSE proche de 0 indique un modèle performant (après
correction).

# 3) Rôle des hyperparamètres
# Le learning rate contrôle la vitesse de mise à jour des poids :
# - Trop faible → apprentissage lent et non convergence
# - Trop élevé → instabilité
# Le nombre d'époques correspond au nombre d'itérations d'entraînement
:
# un nombre insuffisant empêche le modèle d'atteindre une solution
optimale.

# 4) Extraction des coefficients
# Les coefficients (poids w et biais b) sont extraits avec
model.get_weights().
# Ils permettent d'écrire l'équation du modèle et de vérifier la
relation apprise.
# Dans ce TP, on a observé une équation incorrecte au début,

```

puis correcte après ajustement des hyperparamètres.

5) Interprétation des prédictions et évaluation

Les prédictions doivent être comparées aux valeurs réelles pour évaluer le modèle.

Dans le cas initial, les prédictions étaient incohérentes (valeurs négatives).

Après correction, elles deviennent proches des valeurs réelles.

La MSE permet de quantifier cette performance et de valider la convergence du modèle.